

RESEARCH ARTICLE

Software Metric Based Impact Analysis of Code Smells - A Large Scale Empirical Study

Md. Masudur Rahman  | Abdus Satter | Md. Mahbubul Alam Joarder | Kazi Sakib

Institute of Information Technology, University of Dhaka, Dhaka, Bangladesh

Correspondence: Md. Masudur Rahman (bit0413@iit.du.ac.bd)

Received: 14 April 2023 | **Revised:** 14 October 2024 | **Accepted:** 14 December 2024

Funding: Financial support was provided by the Information and Communication Technology (ICT) Division, Ministry of Posts, Telecommunications and Information Technology, Dhaka, Bangladesh; Grant/Fellowship Number: 56.00.0000.052.33.005.21-2, 18-01-2022.

Keywords: code smells | impact analysis | mining software repositories | software metrics

ABSTRACT

Context: Code smells are indicators of poor design and implementation choices that negatively affect software quality and maintainability. Moreover, it is difficult and time-consuming to work with a long list of the code smells, as not all of those smells have equal impact on the system. So, understanding the individual impact of the code smells is significant while performing refactorings on a priority basis.

Objective: Despite significant research efforts aimed at detecting and refactoring these code smells, understanding their individual impact on software quality metrics such as size, complexity, coupling, etc. remains still unclear.

Methodology: To mitigate this research gap, we present an empirical investigation on the impact analysis of code smells based on the 25 software quality metrics such as size, cyclomatic complexity, coupling, etc. To the best of our knowledge, this is the largest empirical study about the impact analysis of code smells with respect to the number of software metrics. Particularly for this study, we identify 13 code smells in 35 open-source software systems, and analyze (1) the relationship between code smells and software metrics, (2) which code smells are highly impactful that affect the metrics, and (3) which impactful smells occur frequently in the systems.

Results: The results show varying degrees of correlation-based impact between specific code smells and software metrics, with some smells showing strong correlations with multiple metrics. Three categories of impact for the code smells have been identified, namely High, Moderate and Low, where Long Method, Anti Singleton, Complex Class, Large Class and Long Parameter List smells have high impact, but their frequencies are not high except Anti Singleton; Refused Parent Bequest, Spaghetti Code and Blob have moderate impact; and rest of the smells have low impact. We also observe that perceptions about the impact of code smells vary from developer to developer and they most cases refactor the smells based on their intuition.

Conclusion: Our findings will help them to refactor the smells on an objective-based instead of an intuition-based, which will be more significant to improve the software quality. For example, refactoring the smells having a high impact on the coupling between objects metric can be an objective. Furthermore, our results will not only assist developers in prioritizing refactoring activities but also provide researchers with valuable insights to innovate tools that prioritize refactoring based on the impact of code smells. These tools will help developers target the most impactful smells and thus enhance the overall quality and maintainability of software systems.

1 | Introduction

Code smells are the symptoms of poor design quality, indicating that a software system is hard to maintain [1, 2]. Moreover, the presence of code smells indicates the possibility of introducing more faults in the long run [3]. To improve the quality of a system, developers identify and remove code smells by following a standard process that is known as refactoring [1]. It is expected that they should identify all the smells and refactor properly to make the system smooth for long-term maintenance. However, due to the release pressure, budget, and time constraint issues, they might not be able to refactor all the smells. Moreover, it is very difficult for them to write code considering all possible smells [4]. Usually, in this case, they apply refactoring on the smells that are well-known to them based on their intuition [5]. Since all the smells do not have the same effect on the decline of software quality, refactoring the low-impactful and leaving the higher impactful smells might not help to ultimately achieve the optimum goal (i.e., making software more maintainable in lesser time). So, a study is required to understand which smells have a higher impact on the software quality that will help developers in their refactoring decisions.

Code smells have negative impact on the quality of a software system that affects its maintainability [6]. Software quality can be measured by software metrics such as size, complexity, coupling, etc. [7, 8]. So, code smells are important design factors that affect the quality metrics of a software system. Minimizing these metrics through removing important or impacted code smells is one of the goals of developers in the maintenance phase of the system. Therefore, it is necessary to identify the relationship between code smells and software metrics so that the metrics impacted by a particular smell can be recognized. Moreover, it is very difficult for the developers to remove or work with all the code smells at a time from a developed system, as it is a huge time-consuming task. It is also not wise to refactor only known or frequent code smells, as not all of the smells have the same impact or importance to be removed. For example, some smells might have less impact on software quality, and also some smells occur with very low frequency, which developers can focus on with less priority, as low frequency means less effect on maintainability [4]. Therefore, it is necessary to identify such code smells having a higher impact on the software quality metrics to be refactored. So, it will help developers to refactor not only the known smells by definition but also the impactful ones, which ultimately improve the design quality of the system.

However, developers usually introduce code smells due to a lack of awareness about design principles and program comprehensibility [1]. So, it is crucial to make developers aware of the presence of code smells, especially those that have a significant impact on the system. Software metrics are important factors to measure the impact of code smells. Therefore, in this study, we define *impactful code smell* as one that affects a software system, particularly in terms of software quality metrics such as size, complexity, coupling, etc. In other words, a code smell is considered highly impactful if it strongly correlates with these metrics, and less impactful if the correlation is weak. If a developer intends to reuse code from an existing system and is aware of the potentially impactful code smells present, they can refactor these smells (if applicable) before integrating the code into their own

system [4]. This proactive approach enhances the maintainability of the system in the long term. Moreover, providing developers with a short list of impactful code smells, as opposed to all possible smells or only the well-known ones, helps them to focus and prioritize effectively during development and maintenance. In practice, developers might sometimes refactor impactful smells by chance through intuition-based refactoring, while others might address less impactful smells, which might not align with the goal of perfective maintenance [3]. The objective of the maintenance activity is to improve code quality in less time through refactoring the impactful smells. Therefore, establishing a clear relationship between code smells and software quality metrics is essential to identify these impactful smells. This relationship also helps them to transform their refactoring activities from intuition-based into objective-based. For example, if the objective is to optimize the *coupling between objects* metric in a system, developers should focus on refactoring those code smells that are related to this specific metric.

Software has two types of quality attributes or metrics: Internal and external software quality metrics [9]. In the literature, many works have covered the impact of code smells on the external software metrics such as maintainability, testability, changeability, etc. [6]. More specifically, the impact of code smells on software maintainability has widely been analyzed by previous works. For instance, Khoma et al. [10] and Palomba et al. [2] investigated the impact of code smells on two maintainability metrics, namely code change-proneness and fault-proneness. Abbes et al. [11] showed that multiple code smells affecting the same source code have a negative impact on program comprehensibility. Moreover, some works have focused on trying to reduce the number of code smells by prioritizing or filtering them [12]. For instance, subjective factors such as developers' perceived criticality [13] and developers' relevance [14] have been used to prioritize code smells. Sae-Lim, Hayashi, and Saeki [15] proposed a technique to prioritize code smells by considering developers' current context. It only facilitates solving code smells in only current context of a system. However, very few researches have been found about their impact on internal software metrics such as size, complexity, coupling, etc. [6]. Also, there is a noticeable lack of knowledge about the identification of impactful code smells based on the internal software metrics that helps prioritizing these smells in the maintenance phase. To mitigate this research gap, we present an empirical study aimed at investigating the relationship between code smells and internal software quality metrics to identify their impact.

In order to conduct the analysis, an empirical study has been carried out on 35 open-source Java systems. From these systems, we have identified 13 types of code smell and 25 internal software metrics to further measure the relationship between them and identify the impactful code smells. For this study, we refer 'code smell type' as 'code smell', that is, we use 'code smell type' and 'code smell' interchangeably. In the context of this paper, the 'impact' of a code smell refers to the relationship based on correlation between the code smell and the software metrics. To the best of our knowledge, this is the largest empirical study investigating the impact or relationship between the presence of code smells and software metrics, particularly in terms of the number of metrics examined. Additionally, we quantify the impact of each code smell based on correlation analysis, which

aids in the prioritization of code smells. We also identify the impact of code smells based on expert developers' perceptions and compare these with the results found from the empirical study. Moreover, we analyze which impactful code smells are most frequent, meaning those that occur most frequently in the systems. The comparative analysis between the impact of code smells and their frequency of occurrences makes the study different from the existing works. Moreover, this comprehensive approach allows us to better understand which code smells have the greatest effect on software quality and how often they appear, providing valuable insights for improving software maintenance and development practices.

The results of our study provide three categories of impact level for the 13 code smells: *High*, *Moderate*, and *Low*. In particular, the results show that four code smells, namely *Long Method*, *Anti Singleton*, *Complex Class*, *Large Class*, and *Long Parameter List*, have a high impact on the systems due to having high correlations with the software metrics. At the same time, their frequencies of occurrences are also high except for *Anti Singleton*, which has a moderate frequency. *Refused Parent Bequest*, *Spaghetti Code*, *Speculative generality* and *Blob* have a moderate impact, and the rest of the smells have a low impact on the metrics. On the contrary, *Many Field Attributes But Not Complex* is the low-impactful code smell having the low frequency. Our results are significant in the sense that code smells are not only highly correlated with the smell-definition-based metrics but also with various other metrics. We also observe that different metrics exhibit varying levels of impact on the code smells. These insights will assist developers in shifting from intuition-based to objective-based refactoring of code smells. For example, if the objective of a refactoring activity is to optimize the *Lack of Cohesion in Methods (LCOM)* metric, the developer should not only refactor *Blob* or *Large Class* smells, but also other relevant smells such as *Anti Singleton*, *Complex Class*, *Long Method*, etc. We also found that the impact of code smells can vary between developers. Therefore, our guidelines will help prioritize refactoring efforts, ultimately improving overall software quality. Additionally, this information can aid research practitioners in developing innovative refactoring and smell prioritization tools based on the impact data.

For the empirical study, we constructed a dataset from 35 selected open-source systems, collecting both code smell and software metric-related information. This dataset is organized into a set of directories containing smelly files for each of the 13 smells separately for each project. The dataset provides a structured foundation for our impact analysis on software quality metrics. By organizing the data in this way, we enable a focused investigation into the correlation between specific code smells and software metrics, allowing for a more granular analysis of their impacts. Moreover, this dataset serves as a valuable resource not only for our study but also for future research. It offers researchers a rich source of information to analyze individual code smells in detail, facilitating deeper exploration into the characteristics and behavior of each smell. Both researchers and developers can leverage this dataset to prioritize code smells with the higher impact on maintainability, complexity, and other critical software attributes, which drive better software development practices.

In summary, this paper makes the following contributions:

- (i) **Quantifying the Impact of Code Smells:** We establish the relationship between code smells and software metrics to quantify the impact of these smells.
- (ii) **Impact Levels of Code Smells:** We categorize code smells into three impact levels *High*, *Moderate*, and *Low*—to help prioritize them. This prioritization can be used to optimize software metrics and enhance software quality.
- (iii) **Comparative Analysis of Frequency and Impact:** We conduct a comparative analysis between the frequency and impact of code smells, demonstrating that developers should focus on those smells that have both high impact and high frequency.
- (iv) **Synthesized Smelly Dataset:** We provide a synthesized dataset containing 74,253 smelly files where each of the 13 smelly files is stored separately for the 35 systems. This dataset can be used by the research community for further studies and to build upon our research.

Structure of the paper: Section 2 discusses the related work about prioritization of code smells and their impact on software systems. Section 3 describes the design of the empirical study, and Section 4 discusses the synthesized dataset generation process. Section 5 presents and discusses the results of the study, while the threats to their validity are discussed in Section 6. Finally, Section 7 concludes the paper, describing the key findings and future direction of the work.

2 | Related Work

After defining 22 types of code smells by Fowler et al. [1], lots of works have been performed on these smells in the literature. Over the past two decades, research on code smells has gained increased interest from both software engineering practitioners and researchers [16]. Most of the works have covered the detection approaches and refactoring of the smells. For example, JDeodorant [17], an Eclipse plug-in, can identify six types of code smells and provide refactoring suggestions using static code analysis, while Moha et al. [18] identified 18 types of smells using the rule card. Palomba et al. [19] used historical information of software releases to detect code smells. Indeed, the various code smells do not have the same effect on software quality, and consequently, they do not all carry the same level of importance. Therefore, several works exist in the literature regarding the impact of code smells on the software quality and their prioritization. This existing literature is discussed in this section.

2.1 | Impact of Code Smells on Software Quality

Code smell is an important factor that affects the quality of a software. In the ISO/IEC 9126 standard (International Organization for Standardization) [20], software quality is defined through a set of attributes (or metrics) used to describe and evaluate the quality of the software. These attributes are typically divided into two categories [9]: *Internal quality attributes*, which can be directly measured from the source code (e.g., lines of code, complexity), and

external quality attributes, which are assessed indirectly based on the behavior and performance of the software (e.g., maintainability, usability). Several studies have been performed to analyze the impact of code smells on software quality attributes.

Internal software metrics are commonly utilized to detect various types of code smells [17, 18, 21]. Sousa, Bigonha, and Ferreira identified five distinct types of code smells using these metrics and explored the relationship between design patterns, and code smells through source code analysis [22]. Their findings revealed that certain design patterns such as adapter, proxy, and strategy, exhibited a high frequency of co-occurrence with the code smells.

Khomh et al. [10] investigated that code smells have a negative impact on two maintainability measures—change-proneness and fault-proneness of classes in a system after analyzing 13 smells considering releases of four open-source systems. They also concluded that some types of code smells, as expected by their definitions, are related to size metric, whereas only size can not explain the classes' higher change- and fault-proneness. In another study, they [23] showed that classes having code smells are significantly more change-prone than classes with no code smells. Some other studies also provide evidence about the positive relationship between code smells and fault-proneness [24, 25]. Saboury et al. [26] also found the negative impact of code smells on the fault-proneness in JavaScript systems. Moreover, Zhong et al. [27] found that security smells have an impact on faults in android applications.

After conducting a large-scale empirical study, Palomba et al. [2] confirmed the findings of the previous studies showing that classes affected by code smells have higher change- and fault-proneness than other classes. They used 13 different types of smells and 30 Java systems for their investigation. They observed that long and complex code smells, such as *Long Method*, *Complex Class*, etc., are more change- and fault-prone. They also showed that classes affected by multiple smells are more critical than classes affected by single one. In another empirical study, Rahman et al. [28] showed that higher frequent code smells such as *Complex Class*, *Large Class* and *Long Parameter List* have a moderate impact on fault-proneness. On the other hand, lower frequency smells such as *Anti Singleton*, *Blob*, and *Class Data Should Be Private* have a high impact on fault-proneness.

Abbes et al. [11] investigated the impact of two code smells, namely *Blob* and *Spaghetti Code*, on program comprehension. The results of their empirical study showed that the occurrence of one code smell in the source code of a system does not have a significant impact on developers' ability to comprehend the code. Instead, a combination of multiple code smells affecting the same source code strongly reduces program comprehensibility. In another study, Rahman et al. [29] showed that feature envy code smells have an impact on software standards called naming conventions, which reflect program comprehension.

There exist several other studies on the interaction among different code smell types on the same code components. For example, Yamashita and Moonen [30] investigated inter-smell relations and their effects on maintenance problems using 13 smells on four Java systems. They showed that developers face more difficulties while working on classes affected by more than one

code smell. In another study, they [30] identified the important maintainability factors, such as inheritance, simplicity, logic spread, etc., from the software maintainer's perspective, those are reflected by code smells. In addition, they [31] investigated inter-smell relations where they used static dependency analysis involving both open-source and industrial systems. They observed that code smell interactions occur not only in the same file but also across coupled files having static dependencies. Also, Palomba et al. [32] conducted an empirical study showing that almost 59% of smelly classes contain more than one code smell type. They observed that method-level smells are responsible for causing the class-level smells. In addition, Martins et al. [33, 34] showed that two types of code smell tend to occur together, and removal of those co-occurrences reduces system complexity significantly.

Sjoberg et al. [35] investigated the impact of 12 code smells on the maintenance effort in software systems. They observed that code smells do not have a significant association with maintenance effort; rather than, often class size has more impact on maintainability. On the other hand, some studies found that code smells related to long and complex source code are generally perceived as design problems by developers, whereas some smells are not [36, 37].

Bavota et al. [5] observed that 42% of refactoring operations are performed on code entities affected by code smells, whereas only 7% actually remove the code smells. In this study, they actually investigated 11 code smells on three open-source systems. In addition, they suggested that developers refactor source code for general improvement of maintainability, whereas software quality metrics might suggest classes as good candidates to be refactored. Moreover, they also observed that refactorings are mainly attributable to a subset of known smells such as *Blob*, *Long Method*, etc. These findings indicate that there needs to be more research to identify the significant software metrics to refactor impactful code smells.

Kaur [6] conducted a systematic literature review and observed that different code smells have different effects on various software quality attributes, and hence the impact of code smells on software quality is not uniform. The study also observed that 92% of the primary studies (PSs) considered external quality attributes such as readability, maintainability, changeability, etc., to investigate the impact of code smells. On the other hand, only 8% of the PSs considered five internal quality attributes, namely size, coupling, cohesion, complexity, and inheritance, to investigate the impact. Therefore, it is necessary to study more about code smells considering the internal quality attributes.

2.2 | Prioritization of Code Smells

Code smells are important design factors, but all of these are not equally important for software quality [6, 14]. Moreover, it is a good practice to prioritize impactful code smells and refactor them first to save time and effort [38]. So, important code smell types are necessary to be identified so that developers can prioritize these for refactoring activities. Therefore, several studies have been conducted to prioritize these code smells.

Vidal et al. [39] proposed a tool called JSpIRIT that prioritizes code smells based on multiple criteria (like the relevance of the smells, history of the system, etc.) that can be customized by developers. In addition, they [14] proposed a semi-automated approach that prioritizes code smells based on three criteria: (i) past component modifications, (ii) important modifiability scenarios for the system, and (iii) relevance of the kind of smell. However, the third criterion is a subjective value that might vary from developer to developer. Several other studies used developers' perceived criticality and preferences to prioritize code smells [13, 40]. However, these approaches require human intervention and might provide a different prioritized set when development personnel are changed.

Sae-Lim, Hayashi, and Saeki [12, 15] proposed a technique to prioritize code smells based on developers' current context. The context can be identified by the change descriptions of an issue that are going to be implemented in a particular milestone. The approach might not be applicable if the issue descriptions are unavailable. In another study, Islam, Al Maruf, and Cerny [41] proposed a strategy to prioritize code smells based on frequently used and change-prone areas of source code in a system by combining business logic, heat map or frequency information, commit history analysis, and severity levels of code smells.

Guggulothu and Moiz [42] proposed an approach that suggests an order for four smells based on the relationship among the smells and relevant static code metrics such as lines of code, coupling, etc. The relevant metrics for each code smell have been identified with the help of a machine learning technique called feature selection. Another study used severity for five code smells to prioritize these using the static code metrics [43]. Fontana et al. [44] used the intensity index for prioritization of six code smells, that facilitates identifying the most severe code smell instances. They defined an intensity index as to quantify how critical each code smell instance is compared to other instances of the same code smell. The intensity index is dependent on the detection rule of code smells and the corresponding metrics distribution. In another study, Oliveira et al. [45] suggested two important heuristics (namely, density and diversity of smells); those are promising to support developers in prioritizing code smells.

From these studies stated above, it has been seen that code smells have a negative impact on software quality. Most of the studies have covered the impact of code smells on external quality metrics such as maintainability, whereas very few studies have focused on internal quality metrics. In addition, there is no consensus on what the important metrics are and how to measure their impact [46]. Therefore, the impact of code smells on software quality with respect to the internal quality metrics is still unclear. Furthermore, it is crucial to prioritize code smells based on their specific impact because not all code smells affect software quality to the same degree. Prioritizing them can help in efficiently addressing the most harmful ones first, thereby improving overall software quality more effectively.

3 | Empirical Study Design

The goal of the study is to assess the impact of 13 code smells based on 25 internal software metrics (also known as internal

quality attributes) by analyzing 35 software systems with the purpose of helping in maintenance activities. It is noted that this is the largest empirical study with respect to the number of internal quality metrics of software systems. More specifically, the study aims at identifying the impactful set of code smells that affects the metrics. It is worth remarking that the term "impactful," when associated with a code smell, refers to the correlation of the code smell with the software metrics. Indeed, some smells might be highly correlated with the metrics but rarely occurred in the systems or vice versa. Therefore, frequency analysis of code smells having their impacts is also another goal of the study. In this section, we discuss the research questions of the study (Sub-section 3.1), selected software systems under the study (Sub-section 3.2), selected code smells (Sub-section 3.3), and software metrics (Sub-section 3.4).

3.1 | Formulating Research Goal

The study aims at identifying the impactful set of code smells based on their relationship with software metrics by answering the following three research questions:

RQ1: What is the relationship between code smells and software metrics?

The research question investigates which software metrics are highly correlated with which code smells by analyzing the correlation between them in the software systems. More specifically, a set of clusters of software metrics for each code smell based on the relationship is to be identified. The outcome will assist software developers to emphasize the smells to be refactored in order to optimize the highly correlated metrics.

RQ2: What is the impactful set of code smells?

Code smells that have a higher correlation with software metrics are highly impactful. The list of these highly impactful smells should be given higher priority during maintenance activities, especially when performing refactorings to improve software quality. The target of this research question is to categorize the code smells into three impact levels—(High, Moderate, and Low)—based on the relationship analysis in RQ1.

RQ3: Which frequent code smells are highly impactful?

Not all of the highly impactful code smells might be important based on their frequency of occurrences in the systems. It could be possible that some high-impactful code smells could occur with very low frequency, and hence these could be avoided in the maintenance phase. Similarly, low-impactful smells with high frequency could also be avoided. On the other hand, high-impactful smells having higher frequency are important since these actually affect the maintenance activities. So, only impact analysis of the smells is not enough, but frequency analysis is also important. Therefore, the objective of this research question is to compare the impact and frequency of various code smells. By doing so, we aim to identify the code smells that occur frequently and have a significant impact on software quality. These are the code smells that should be prioritized during the maintenance phase to ensure the most effective improvements in software quality.

3.2 | Systems Under Study

The context of the study consists of (i) software systems, (ii) code smells, and (iii) software metrics. In order to conduct the empirical study and answer the research questions, we have analyzed 35 Java systems belonging to two major ecosystems: Apache¹ and Eclipse². The systems of these ecosystems are well-known

in the code smell research domain [2, 13]. Table 1 summarizes the analyzed systems, the latest stable version of the systems up to this study, their size in terms of number of classes (NOCs), number of methods (NOMs), and lines of codes (LOCs), and history of the projects as starting to ending year, including their duration. These systems have been selected because they are in different sizes (from 8,069 to 1,762,363 LOCs), large enough to

TABLE 1 | Software systems involved in the study.

Id no.	Project	Version	# Classes	# Methods	LOCs	Project history (Start to end year, duration in years)
1	Activemq	5.17.0	4,747	42,349	421,979	2005 to 2022, 17
2	Ant	1.9.9	1,595	13,768	138,391	2000 to 2013, 13
3	Ant-ivy	2.5.0	814	7,540	76,678	2005 to 2019, 14
4	ArgoUML	0.35.1	2,096	17,955	177,402	1998 to 2014, 16
5	Cassandra	4.0.5	8,101	76,237	1,483,583	2009 to 2021, 12
6	Cayenne	4.1	6,656	41,339	485,781	2007 to 2020, 13
7	CDT	10.6.2	4,254	26,883	267,165	2002 to 2022, 20
8	CXF	3.5.3	11,109	103,490	1,120,148	2008 to 2021, 13
9	DeepLearning4j	1.0.0	9,630	59,725	698,622	2019 to 2022, 3
10	Drill	1.10.0	6,414	60,203	533,478	2012 to 2017, 5
11	Eclipse Core	R4.9	4,391	32,977	305,762	2001 to 2018, 17
12	ElasticSearch	8.3.3	23,187	154,096	2,097,667	2010 to 2022, 12
13	Freemind-mmxx	1.0.0	744	6,804	63,933	2011 to 2013, 2
14	Hadoop	3.3.4	17,393	143,785	1,762,363	2009 to 2022, 13
15	HBase	3.0.0	9,457	71,619	823,856	2007 to 2021, 14
16	Hive	3.1.3	14,466	107,711	1,257,002	2008 to 2018, 10
17	Hsqldb	2.7.0	952	13,874	208,333	2007 to 2022, 15
18	Incubator-livy	0.7.0	201	811	8,069	2015 to 2020, 5
19	Jackrabbit	2.9.0	3,209	29,053	332,826	2004 to 2014, 10
20	JBoss-modules	2.0.3	301	1,956	20,586	2010 to 2022, 12
21	Jena	4.5.0	7,077	66,138	576,575	2012 to 2022, 10
22	JFreeChart	1.5.3	955	11,336	136,786	2007 to 2021, 14
23	JHotDraw	9.0	768	7,630	80,440	2000 to 2020, 20
24	Karaf	4.4.1	1,760	10,332	126,431	2007 to 2022, 15
25	Lucene	9.3.0	7,501	52,968	827,867	2001 to 2022, 21
26	Mahout	14.1	1,551	9,552	111,437	2008 to 2019, 11
27	Nutch	2.4	519	3,136	38,268	2005 to 2019, 14
28	Opennlp	1.9.4	1,075	5,303	75,724	2010 to 2021, 11
29	Pig	0.9.2	2,222	15,433	231,371	2007 to 2012, 5
30	Poi	5.2.2	4,291	40,032	413,657	2002 to 2022, 20
31	Qpid	0.32	3,374	29,189	327,821	2006 to 2014, 8
32	Struts	6.0.0	2,378	18,640	186,595	2006 to 2022, 16
33	Tomcat	10.0.22	3,989	33,433	355,081	2006 to 2022, 16
34	Wicket	7.2.0	4,402	20,678	215,286	2004 to 2016, 12
35	Xerces	2.9.1	800	9,809	131,326	1999 to 2006, 7
Total		—	172,379	1,345,784	16,118,289	—
Average		—	4,925	38,450	460,522	12.5

have an average NOCs of 4,925; NOMs of 38,450, and LOCs of 460,522. In addition, the selected systems have an average duration of 12.5 years and cover different application domains such as *ArgoUML* – a UML-based system modeling tool, *Eclipse* – an IDE, *Lucene* – a search manager, *Xerces* – an XML parser, etc.

3.3 | Code Smells Selection

To carry out the investigation, a list of 13 code smells has been selected. These 13 code smells, including a short description, are listed in Table 2. An existing tool named DÉCOR [18] has been used for the detection of these smells from the selected systems because the tool shows good accuracy and has been widely used in the previous studies [2, 10, 47].

We choose these code smells for the study because (1) these are representative of various object-oriented design issues such as inheritance, polymorphism, encapsulation, etc., (2) the frequencies of these smells are great in numbers to conduct the empirical study, and (3) their frequencies are almost similar between open-source and industrial systems [4].

3.4 | Software Metrics Selection

Software quality is an important factor in measuring how much a software system is maintainable. In ISO/IEC 9126 set by the International Organization for Standardization [20] standard,

software quality is defined by a series of attributes or metrics that describe and assess the quality of a software system. These quality attributes are divided into two categories [9]: Internal quality attributes and external quality attributes. Internal quality attributes are those that can be directly measured from the software's source code, such as lines of code. External quality attributes, on the other hand, are measured indirectly from the software, such as understandability. However, this study focuses solely on internal quality attributes, leaving external quality attributes beyond its scope.

In this research, we examine 25 internal quality attributes, which we refer to as internal software metrics or simply software metrics for ease of reading. These metrics are computed by a well-known tool designed for research purposes, called Understand³ [8]. The metrics according to the name provided by the tool, their abbreviations, and short descriptions are shown in Table 3. For clarity and readability, these abbreviations are used in the results section.

We have selected these metrics for our study for several reasons:

1. **Comprehensive Coverage of Design Factors:** The chosen metrics encompass six critical design factors of a software system: complexity, coupling, cohesion, abstraction, encapsulation, and documentation [8]. These factors are essential in providing a well-rounded assessment of software quality.

TABLE 2 | Code smells involved in the study.

Id No.	Code smell	Description
1	Anti singleton (AS)	A class that provides mutable class variables, which consequently could be used as global variables.
2	Base class should be abstract (BCSBA)	An inheritance tree contains roots that are not abstract – only the leaves should be concrete.
3	Blob	A class that is too large and not cohesive enough, that monopolises most of the processing of a system, takes most of the decisions, and is associated to data classes.
4	Class data should be private (CDSBP)	A class that exposes its fields, thus violating the principle of encapsulation or data hiding.
5	Complex class (CC)	A class that contains (at least) one large and complex method, in terms of cyclomatic complexity and LOCs.
6	Large class (LC)	A class that is large in size in terms of LOCs.
7	Lazy class (LzC)	A class that has few fields and methods (with little complexity), and that does not do too much in the system.
8	Long method (LM)	A method that is very large in size and complex.
9	Long parameter list (LPL)	A method that contains a long list of parameters.
10	Many field attributes but not complex (MFABNC)	A class that is not complex but has too many fields those are public.
11	Refused parent bequest (RPB)	A class that inherits functionalities from its parent but never uses these, thus violating polymorphism.
12	Spaghetti code (SC)	A class without structure that declares long methods without parameters, thus prevents polymorphism.
13	Speculative generality (SG)	A class that is defined as abstract but it has very few children, which does not make use of its methods.

TABLE 3 | Software metrics involved in the study.

Id no.	Software metric	Description
1	CountDeclMethod, NOM	Number of local methods
2	CountDeclMethodDefault, NDM	Number of local default methods
3	CountDeclMethodPrivate, NPriM	Number of local private methods
4	CountDeclMethodProtected, NProM	Number of local protected methods
5	CountDeclMethodPublic, NPM	Number of local public methods
6	CountStmt, NOS	Number of statements
7	CountStmtDecl, NDS	Number of declarative statements
8	CountStmtExe, NExS	Number of executable statements
9	SumCyclomatic, CC	Sum of cyclomatic complexity of all nested methods
10	LCOM	Lack of cohesion in methods
11	Depth of inheritance tree, DIT	Maximum depth of a class in inheritance tree
12	IFANIN	Number of immediate base classes
13	Coupling between objects, CBO	Number of other classes to which a particular class is coupled
14	Number of children, NOCh	Number of immediate subclasses
15	Response for a class, RFC	Number of methods including inherited ones
16	Number of instance methods, NIM	Number of methods defined in a class that are only accessible through an object of that class
17	Number of instance variables, NIV	Number of variables defined in a class that are only accessible through an object of that class
18	Weighted methods per class, WMC	Sum of the complexities of all class methods
19	Classes, NOC	Number of Classes
20	Files, NOF	Number of Files
21	Lines, NL	Number of all lines
22	Lines blank, BLOC	Number of blank lines of code
23	Lines of code, LOC	Number of lines containing source code
24	Lines comment, NCL	Number of comment lines of code
25	RatioCommentToCode, RCTC	Ratio of comment lines to code lines

2. **Measurement of Software Maintainability:** These design factors are capable of measuring various aspects of software maintainability, as outlined in the study by Zhang et al. [8]. This ensures that our analysis is robust and relevant to the practical needs of software maintenance.

3. **Representation of Different Levels:** The metrics are divided into two groups based on their levels. Metrics Id No. 1 to 19 represent class-level (or file-level) metrics, while metrics 20 to 25 represent project-level metrics. For the purpose of our analysis, we have aggregated the class-level metrics to create project-level metrics, allowing us to maintain a consistent level of analysis across the dataset.

4 | Synthesized Dataset Generation

In order to analyze the impact of each code smell on the software metrics, we need to create a dataset from the selected systems from which we can collect information about the smells and metrics. This dataset consists of a set of directories containing smelly files for each of the 13 smells separately for each of the 35 projects. In this study, we define a 'smelly file' as—a file is called a smelly

file for a code smell type if it contains that type of code smells. For example, if a file contains a *Long Method* code smell, then this file is called a 'long method smelly' file. It is noted that we consider any level of smells such as class method, or block level, as a smelly file. Because most of the code smells are introduced at the time of file creation [48] and smelly files are more likely to undergo changes [49]. Therefore, developers write and manage code at the file level, making it a base unit of analysis. Moreover, when developers examine code smells, they tend to look for issues that affect the entire file, rather than just specific methods or blocks of code.

In particular, the dataset contains the following three levels of directories. The generic structure of the dataset directory is shown in Figure 1, where PD_i ($i = 1, 2, 3, \dots, N$) is the i th project directory, SD_j ($j = 1, 2, 3, \dots, M$) is the j th smelly directory, and in this study $N = 35$, $M = 13$.

- (i) **Level 0—Master Directory:** It is the name of the root directory that contains project directories. In our study, it has 35 (N) directories for the corresponding 35 project names.
- (ii) **Level 1—Project Directory (PD):** For each of the 35 project directories in the master directory, it contains 13 (M)

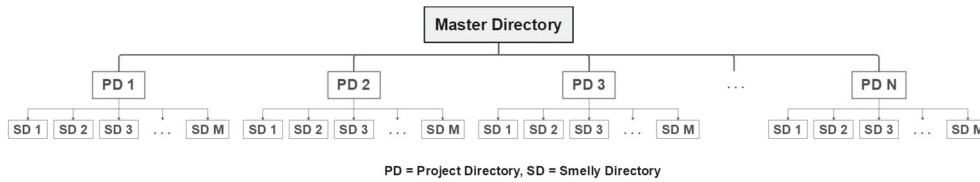


FIGURE 1 | Generic structure of the dataset.

directories for the corresponding 13 code smells. These directories are used to store smelly files of the corresponding code smell directory of the corresponding project.

- (iii) *Level 2–Smelly Directory (SD)* It is the last level of the directories. For each of the 13 smelly directories, it contains the corresponding smelly files for a particular project.

For example, the *Activemq* project has 13 smelly directories in which *Anti Singleton* smelly files reside in the *AntiSingleton* directory, *Long Method* smelly files are in the *LongMethod* directory, and so on. In this way, we have $35 \times 13 = 455$ smelly directories in total for all the 35 systems. We refer to this whole dataset as a 'synthesized dataset for code smells'. The dataset is given here⁴ so that more research and analysis about code smells can be performed.

The dataset generation is a two-step process: (1) code smell detection and (2) smelly directory generation. These steps are discussed in the following subsections.

4.1 | Code Smell Detection

To create the synthesized dataset, all the 13 code smells are needed to be detected for each of the 35 systems. A popular smell detection tool, DÉCOR [18] has been used for this purpose. This tool detects the code smells for the 35 systems as well as provides a list for each of the smells, which is unstructured since it contains many information, including the corresponding smelly file names. We then organize the unstructured smelly lists to make these structured by filtering only the smelly file names for each smell type. Therefore, we can use these structured files to generate the smelly directories having the smelly files.

We have developed a python script that makes this structuring process of filtering easier, as manual work requires a huge time. However, we also manually validated the output of the structured files to avoid whether there exist any missing smelly file names. It is noted that all the structured files containing the smell names as well as scripts related to this dataset generation process are provided⁵.

4.2 | Smelly Dataset Generation

According to the structure given in Figure 1, we have first created 35 empty directories (folders) for the corresponding 35 systems in the *master directory*. These are the *project directory* of our dataset structure. Then, in each of the project directories, we have created another 13 empty *smelly directories*.

Now, we need to fill up the empty smelly directories with the corresponding smelly files. Since there exist a lot of smelly files for

each of the systems, it will be a hectic and huge time-consuming task to manually find and copy smelly files. For this, we have written a bash script to find and copy the smelly files from the structured files of smell names from a project automatically. For each smell of each project, we have copied the actual smelly *Java* files into the corresponding smelly directory from the corresponding project using this script. However, we have also manually validated the files in the smelly directories and smelly file names to avoid inconsistencies. For example, we have resolved missing files manually; we have resolved duplicate file names of a project as *same_filename*, *same_filename2*, and so on. It takes 25 min on average for one system to finding and copying the 13 types of smelly files automatically by the script, excluding the manual intervention. That is, for each smell of each systems, it takes less than two (25 divided by 13) minutes. So, for the 35 systems, it requires around $25 \times 35 = 875$ min or 14 h 35 min. In this way, we have generated the synthesized dataset separately and stored project-wise smelly files.

The hard disk space (or size) for the synthesized smelly dataset is 1.35 GB (only *Java* files), whereas the space for the actual (or raw) 35 systems is 2.08 GB including *non-Java* files. It is noted that the synthesized dataset contains 74,253 smelly files in total. We have also observed that there is a small difference (only-0.73 GB) between the sizes of these two types of datasets. This is because multiple smelly directories of a project contain several of the same files because these same files contain multiple types of code smells. For example, file A of a project contains two smells-long method and blob, and hence this file, A will be in both smelly directories of that project. Therefore, in the smelly dataset, the number of total files of a project increases more than the unique number of files.

We use the dataset for analyzing impact of code smells on various software metrics in this study. Researchers can use this dataset to investigate more about the code smells individually, their impact on different software aspects such as code comprehensibility, maintainability, etc. Refactoring and smell detection tools can also be validated for each particular type of smell using the dataset.

5 | Data Analysis and Results

In this paper, a large-scale empirical study has been conducted on 35 open-source software systems to understand the impact and frequency of 13 code smells. Besides, relationships between the prevalence of code smells and software metrics have been investigated. A comparative analysis between the impact and frequency of the smells has also been conducted. The detailed result analysis is explained as follows.

5.1 | Relationship Between Code Smells and Software Metrics [RQ1]

The aim of the research question (RQ1) is to identify the relationship between each code smell and each software metric using correlation analysis. Based on the correlation strength, software metrics are clustered for each individual smell. Moreover, based on the overall relation with respect to all code smells, the metrics are categorized into three impact levels – *High*, *Moderate* and *Low* impact. This overall relationship is referred to as the ‘impact level’ of the metrics in this study. Also, in this study, we define ‘smell frequency’ as the number of occurrences of a code smell in a system.

5.1.1 | Data Analysis Process for RQ1

1. Smell Frequency Calculation

To conduct the study, we have calculated the smell frequency for each of the 13 code smells from the 35 software systems using the DÉCOR tool. That is, in this step, we have 13 types of smell frequency for each of the systems, or, in other words, we have 35 smell frequencies for each code smell considering all the systems.

2. Software Metric Calculation

We have used the Understand tool in the synthesized dataset of the smelly files to calculate the 25 software metrics for each code smell (from the corresponding smelly folder) of each of the 35 systems. Here, we have calculated these metrics at the file level for each smelly folder of each system in the dataset. Then, we have summed up the metric scores of the files of each smelly folder individually to get the smell level metric scores of each system. Thus, we will be able to get the 25 metric scores for each of the 13 smells in the 35 systems. For example, we have calculated the *LOC* scores for each file in the *LargeClass* smelly folder of a system, then summed up the scores of all the files in this smelly folder for that system, and thus got the *LOC* score for the *LargeClass* smell of the system. In this way, we have 35 scores of each software metric for each code smell, considering all the 35 software systems.

3. Correlation Coefficient Measurement

Up to this step, we have collected data on 13 different types of code smell, including their frequencies and 25 software metric scores for each of the 35 software systems in our study. For each code smell, we have calculated the Spearman’s Rank Correlation Coefficient [50] between the smell frequencies and the software metric scores for all 35 systems. We have chosen Spearman’s rank correlation because it is a non-parametric measure, making it suitable for our datasets that do not follow a normal distribution.

For example, we have examined the frequency of the *Large Class* code smell in each system as our *x* variable. Correspondingly, we have taken the *Lines of Code (LOC)* metric scores of the classes identified as *Large Class* in those systems as our *y* variable. We have then calculated the correlation coefficient to determine the relationship between these two variables. It is noted that, to

calculate the correlations, we have used a high-level data analysis language called R [51].

This method has been repeatedly used for all the 25 software metrics for each code smell. Therefore, we have computed 25 correlation coefficients for each of the 13 code smells, resulting in a total of $13 \times 25 = 325$ correlation pairs. This comprehensive analysis of the correlation helps us understand which metrics are significantly impacted by each code smell and which ones remain unaffected. This insight is significant for identifying potential areas for code quality improvement and prioritizing refactoring efforts.

To understand the correlation strength of a variety of degrees, the correlation scores have been grouped into 5 categories according to the approach by Schober, Boer, and Schwarte [52]. According to them, the range 0.00–0.10 represents negligible correlation, 0.10 to 0.39 defines weak correlation, 0.40 to 0.69 indicates moderate correlation, 0.70 to 0.89 denotes strong correlation, and 0.90 to 1.00 is interpreted as very strong correlation. The categories have been further extended to five more categories to understand the negative correlation between occurrences of code smells and software metrics. The ranges are 0.00 to –0.10, indicating negatively weak correlation, –0.10 to –0.39, representing weak; –0.40 to –0.69, moderate, –0.69 to –0.90, strong, and –0.90 to –1.00, very strong negative correlation. These categories are significant to understand the relationship level of each smell with the metrics.

5.1.2 | Result of RQ1

1. Correlation Categories

We have analyzed the correlation scores to identify the relationship between code smells and software metrics. Tables 4–6 show the relationship of each code smell with the 25 software metrics based on correlation strength. Based on the correlation categories discussed earlier, we have grouped the software metrics into these categories for each code smell as shown in the tables. For example, *Anti Singleton* smell is ‘very strongly’ correlated with 22 software metrics (category 0.90 to 1.00), ‘strongly’ correlated with only 2 metrics (category 0.70 to 0.89), and ‘negatively weakly’ correlated with only 1 metric (–0.10 to –0.39). We refer to these correlation categories for a code smell as a cluster of metrics for that particular smell. So the above example is the cluster of the metrics for the *Anti Singleton* smell. Similarly, Tables 4–6 show the clusters (each smelly column in the tables) for the 13 smells, which represent the strength between each smell and each metric. However, the detail about the raw correlation scores is given in Table A1 in the appendix section.

However, the tool DÉCOR detects code smells that rely on the software metrics using a set of rules called rule cards [18]. Therefore, we have provided a list of the metrics⁶ used by the tool for each code smell as shown in Table 7. The list is useful to understand the correlation of the code smells with the software metrics that are not affected by the smell-detection metrics of the tool.

2. Discussion and Examples

For example, Table 5 shows the clustered metrics with the level of correlation for the code smell *Long Method* (column 5). From

TABLE 4 | Cluster of software metrics based on *AS*, *BCSBA*, *Blob* and *CDSBP*.

Correlation	Anti singleton	Base class should be abstract	Blob	Class data should be private
0.90 to 1.00	NOM, NPrIM, NProM, NPM, NSM, NDM, NExM, CC, LCOM, DIT, IFANIN, CBO, RFC, NIM, NIV, WMC, NOC, NOF, NL, BLOC, LOC, NCL		NOM, NPM, NSM, NDM, CC, LCOM, DIT, IFANIN, CBO, RFC, NIM, NIV, WMC, NOC, NOF, NL, BLOC, LOC	NOM, NProM, NPM, NSM, NDM, DIT, IFANIN, NOC, NOF, Lines Blank
0.70 to 0.89	NDM, NOCh	NOM, NPrIM, NPM, NSM, NDM, NExM, CC, LCOM, DIT, IFANIN, CBO, NOCh, RFC, NIM, NIV, WMC, NOC, NOF, NL, BLOC, LOC, NCL	NPrIM, NProM, NExM, NCL	NDM, NPrIM, NExM, CC, LCOM, CBO, NOCh, RFC, NIM, NIV, WMC, NL, LOC, NCL
0.40 to 0.69		NDM, NProM, RCTC	NDM, NOCh	
0.10 to 0.39			RCTC	
−0.10 to −0.39	RCTC			RCTC

TABLE 5 | Cluster of software metrics based on *CC*, *LC*, *LzC*, *LM* and *LPL*.

Correlation	Complex class	Large class	Lazy class	Long method	Long parameter list
0.90 to 1.00	NOM, NPrIM, NProM, NPM, NSM, NDM, NExM, CC, LCOM, DIT, IFANIN, CBO, NOCh, RFC, NIM, NIV, WMC, NOC, NOF, NL, BLOC, LOC	NOM, NPrIM, NProM, NPM, NSM, NDM, NExM, CC, LCOM, CBO, RFC, NIM, NIV, WMC, NOC, NOF, NL, BLOC, LOC		NOM, NPrIM, NProM, NPM, NSM, NDM, NExM, CC, LCOM, DIT, IFANIN, CBO, NOCh, RFC, NIM, NIV, WMC, NOC, NOF, NL, BLOC, LOC	NOM, NPrIM, NProM, NPM, NSM, NDM, CC, LCOM, DIT, IFANIN, CBO, NOCh, RFC, NIM, NIV, WMC, NOC, NOF, NL, BLOC, LOC
0.70 to 0.89	NDM, NCL	NDM, DIT, IFANIN, NOCh, NCL		NDM, NCL	NDM, NExM, NCL
0.40 to 0.69			NOM, NDM, NPrIM, NProM, NPM, NSM, NDM, NExM, CC, DIT, IFANIN, CBO, NOCh, RFC, NIM, NIV, WMC, NOC, NOF, NL, BLOC, LOC, NCL		
0.10 to 0.39			LCOM		
−0.10 to −0.39	RCTC	RCTC	RCTC	RCTC	RCTC

the table, it can be seen that out of 25 metrics, only one metric, namely *RatioToCommentCode*, (*RCTC*), is negatively correlated with the frequency of the smell, although the correlation is weak. By further analyzing the systems, we have observed that this is because developers might write comments to make short methods understandable. *CountDeclMethodDefault*, (*NDM*) and *LinesOfComment*, (*NCL*) are strongly correlated with the smell, whereas other metrics 22 have metrics very strong correlation with the smell. This could be because, by definition *Long Methods* are large in size and so are strongly correlated with the size-related metrics such as *NOM*, *NOF*, *NSM*, etc. Moreover, the smell is also strongly correlated with other significant

metrics such as *LCOM*, *DIT*, *CBO*, etc. We have got this type of observation for other smells too. These observations make the study significant in the sense that the study not only shows the high relationship with the smell-definition-based metrics, but also provides other important metrics that developers should focus on to improve software quality. *Long Method* has also been found in RQ2 (Sub-section 5.2) as one of the high-impactful code smells in our study.

For *Base Class Should be Abstract*, no negatively correlated metrics have been seen. In addition, no metric under the study has been found that has very strong correlation value for the

TABLE 6 | Cluster of software metrics based on *MFABNC*, *RPB*, *SC* and *SG*.

Correlation	Many field attributes but not complex	Refused parent bequest	Spaghetti code	Speculative generality
0.90 to 1.00	NOM, NSM, NDM, DIT, IFANIN, NOCh, RFC, NIM, WMC, NOC, NOF, NL, BLOC, LOC, NCL	NOM, NPM, NSM, NDM, NExM, CC, LCOM, DIT, IFANIN, NOCh, RFC, NIM, NIV, WMC, NOC, NOF, NL, BLOC, LOC, NCL	NOM, NPM, NSM, NDM, NExM, CC, LCOM, DIT, IFANIN, RFC, NIM, WMC, NOC, NOF, NL, BLOC, LOC, NCL	NOM, NPM, NSM, NDM, LCOM, DIT, IFANIN, RFC, NIM, NIV, WMC, NOC, NOF, NL, BLOC, LOC, NCL
0.70 to 0.89	NPriM, NPM, NExM, CC, LCOM, NIV, RCTC	NPriM, NProM, CBO	NDM, NPriM, NProM, CBO, NIV	NPriM, NProM, NExM, CC, CBO, NOCh, RCTC
0.40 to 0.69	CBO	NDM, RCTC	NOCh, RCTC	NOMDefault
0.10 to 0.39	NDM, NProM			

TABLE 7 | Software metrics used in DÉCOR for each code smell.

Code smell	Metrics used in DÉCOR
AS	Global variable
BCSBA	Depth of inheritance tree (DIT), Number of children (NOCh)
Blob	Number of class attributes (NAD), Number of class methods (NMD), Lack of cohesion in methods (LCOM5)
CDSBP	Public field
CC	McCabe cyclomatic complexity
LC	Number of class attributes (NAD), Number of class methods (NMD), Lack of cohesion in methods (LCOM5)
LzC	Weighted methods per class (WMC), Number of class attributes (NAD), Number of class methods (NMD)
LM	Method LOC
LPL	Number of parameters (NOParam)
MFABNC	Weighted methods per class (WMC), Number of class attributes (NAD)
RPB	Inherited from its parents (IR)
SC	Method LOC, Number of parameters (NOParam), Depth of inheritance tree (DIT)
SG	Number of children (NOCh)

occurrences of the smell. This indicates that this smell has lower impact on the software metrics than other smells. For the existence of *Blob*, we have noticed variety of correlations for different metrics; for example, *NDM* and *NOCh* are moderately correlated, and *NPrim*, *NProM*, *NExM*, and *NCL* are highly correlated with the presence of the smell. Other metrics have very strong correlation, but only *RCTC* has a very weak impact on the occurrences of the smell. This represents the moderate impact on the systems. We found a very strong and high relationship for the metrics with the smell *Class Data Should be Private* except *RCTC*, which contributes negatively. For *Complex Class*, *Large Class*, *Long Method*, and *Long Parameter List*, we have observed that almost the same set of metrics has very strong correlation values for having the smell. However, we have not found any metric having strong correlation for the presence of the smell *Lazy Class*. It is the only code smell in our study where the metrics moderately reflect their influences for the presence of the smell. From our observation, by definition, the smell contributes less to systems, and hence no strong relationship is found from the smell with

the software metrics. Quite the same list of metrics can be seen for the existences of the smells—*Many Field Attributes But Not Complex*, *Refused Parent Bequest*, *Spaghetti Code* and *Speculative Generality*. It is interesting that, *RCTC* has a negative correlation with most of the smells, indicating that comments might help in the reduction of the smells.

3. Impactful Software Metrics Identification

To identify the impactful software metrics, we have further investigated the degree of correlation for all the metrics with the frequencies of code smells. The correlation scores for each of the smells with respect to the metrics are squared individually, which is formally known as the coefficient of determination (r^2). The reason for squaring the correlation scores is that it allows us to consider both the positive and negative impacts of the metrics. Next, we have calculated the average r^2 (*Impact Score*) scores for each metric across all 13 code smells, assuming equal importance of each code smell. Because identifying impacts or weights is the

purpose of this research and assigning weights to the smells is the future scope of our research. This calculation has been performed using the following formula (Equation 1):

$$\text{Impact Score} = \frac{\sum_{i=1}^N r_i^2}{N} \quad (1)$$

where r_i is the correlation score between the metric and each code smell, and N is the total number of code smells (in this case, $N=13$).

In this study, this average r^2 , referred to as the 'impact score' or simply 'impact,' represents the overall effect of a particular metric across all code smells. In this way, the impact scores for all the 25 software metrics have been measured.

Afterward, we categorized the metrics into three levels of impact: *High*, *Moderate*, and *Low*. This categorization is based on statistical quartiles, as described by Gupta and Kapoor [50]. We used the first quartile (Q1), the second quartile (Q2), and the third quartile (Q3) to determine these levels of impact. Specifically, we defined the impact levels as follows:

1. *High Impact*: Metrics with r^2 values greater than or equal to Q3, representing the top 25% of the r^2 scores.
2. *Low Impact*: Metrics with r^2 values less than or equal to Q1, representing the bottom 25% of the r^2 scores.
3. *Moderate Impact*: Metrics with r^2 values between Q1 and Q3, representing the middle 50% of the r^2 scores. We have combined Q2 and Q3 [53] due to the relatively low number of scores in these quartiles and the minimal difference between the scores in these ranges.

In this approach, the *High* impact level encompasses the metrics with the highest r^2 values, indicating a strong correlation and significant impact. The *Low* impact level includes the metrics with the lowest r^2 values, indicating a weak correlation and minimal impact. The *Moderate* impact level includes the metrics with intermediate r^2 values, indicating a moderate level of impact. We have combined Q2 and Q3 [53] due to the relatively low number of scores in these quartiles and the minimal difference between the scores in these ranges. This adjustment ensures a more balanced and meaningful categorization. This categorization provides valuable insights into which software metrics are most affected by code smells, helping to identify the key areas for potential improvements in software quality and maintainability.

Table 8 presents a comprehensive list of metrics along with their corresponding average r^2 scores (referred to as impact scores) and their impact levels. The table reveals that 11 out of the 25 metrics are highly correlated with the occurrence of code smells. Additionally, 6 metrics exhibit a moderate impact level, while the remaining 8 metrics are considered to have a low impact. The metric *Number of Files (NOF)* has the highest r^2 value of 0.093, making it the most impactful metric. This indicates that an increase in the number of source files leads to more source code being added by developers, thereby raising the likelihood of introducing code smells. This trend is also evident in other size-related metrics with high impact, such as *Number of Classes (NOC)*, *Lines of Code (LOC)*, *Number of Methods (NOM)*, etc.

TABLE 8 | Impact of software metric based on code smells.

Software metric	Impact score	Impact level
NOF	0.93	High
NOC	0.88	
DIT	0.86	
NDS	0.85	
IFANIN	0.85	
NOS	0.83	
BLOC	0.82	
NOM	0.82	
LCOM	0.82	
NL	0.82	
LOC	0.82	
WMC	0.81	Moderate
NIM	0.80	
NPM	0.80	
RFC	0.79	
NExS	0.78	
CC	0.77	
NIV	0.76	
NCL	0.76	
NPriM	0.69	
CBO	0.68	
NProM	0.64	
NOCh	0.64	Low
NDM	0.46	
RCTC	0.21	

Consequently, developers should prioritize these metrics to enhance software quality. Moreover, two inheritance-related metrics, *Depth of Inheritance Tree (DIT)* and *Number of Immediate Fan-in (IFANIN)*, also show a high impact. Developers should pay attention to these metrics as well. Conversely, the metric for comments (*Ratio of Comment to Code, RCTC*) has the lowest impact on the system. This is because comments serve as documentation to make the source code more understandable, rather than contributing to code complexity or other detrimental attributes. Metrics related to code complexity, such as *Weighted Methods per Class (WMC)* and *Cyclomatic Complexity (CC)*, have a moderate impact on the presence of code smells. Generally, an increase in the values of highly impactful metrics is associated with a higher likelihood of introducing code smells like *Complex Class*, *Long Method*, *Refused Parent Class Bequest*, etc.

The difference in impact scores between the adjacent levels seems very small. To determine whether these differences are statistically significant, we have conducted hypothesis testing using the statistical T-test [50]. Here are the hypotheses for our tests:

- *Null hypothesis (H_0): There is no significant difference in impact scores between adjacent impact levels (e.g., High vs. Moderate, Moderate vs. Low).*

- *Alternative hypothesis (H_1): There is difference in impact scores between adjacent impact levels (e.g., High vs. Moderate, Moderate vs. Low).*

We have carried out the one-tailed T-test (r^2 is a positive value) to compare the impact scores between the following pairs of impact levels: High vs. Moderate and Moderate vs. Low. Results of the hypothesis testing are shown below:

- *High vs. Moderate Impact Levels:* p-value = 0.001 (which is less than the significance level of 0.05)
- *Moderate vs. Low Impact Levels:* p-value = 0.002 (which is also less than the significance level of 0.05)

In both cases, the p-values are well below the 0.05 threshold, where we consider a 5% level of significance. Therefore, we reject the null hypothesis (H_0), indicating that the differences in impact scores between the adjacent levels are statistically significant. The results show that there is a significant difference in impact scores between the *High* and *Moderate* levels, as well as between the *Moderate* and *Low* levels. These findings imply that the impact scores at different levels are indeed distinct and meaningful, providing a valid basis for categorizing the software metrics into *High*, *Moderate*, and *Low* impact levels. Consequently, this analysis confirms that the impact scores serve as a discriminating factor, effectively distinguishing between the different levels of impact.

These findings provide valuable insights for developers, enabling them to optimize the source code based on the impact of various metrics while maintaining the system. By focusing on the most influential metrics, developers can improve software quality and reduce the occurrence of code smells.

4. Implication of the Result

Overall, the results show that many of the smells are highly correlated with most of the metrics, indicating that code smells are harmful for the quality of a software system with respect to the internal software metrics. This means that as the frequencies of the smells increase, these metrics' scores also increase, and the overall quality of the system decreases. So, our results indicate that developers should be careful about the smells that are highly correlated with the metrics.

The correlation categories assist software developers to identify the impactful metrics associated with a particular code smell. It does not only show high relations of a code smell with the smell-definition-based metrics, but also provides other significant and higher relevant metrics associated with the smell. As a result, this contribution helps the developers to refactor code smells with an objective-based rather than traditional intuition and general code improvement-based refactoring approach. For instance, while refactoring a code smell, they will be able to learn about which metrics will get optimized. Similarly, if their objective is to optimize one or more certain significant metrics according to a system necessity, they will be able to get insight about certain impactful and relevant code smells on which they should focus. For example, if a developer wants to optimize the metric *Coupling Between Objects*, (CBO) from a system, s/he might focus

Summary for RQ1. Many of the code smells, such as *Anti Singleton*, *Blob*, *Complex Class*, *Large Class*, *Long Method* and *Long Parameter List* are strongly correlated with most of the software metrics, especially related to the size (e.g., number of files, methods, classes, etc.). Our finding is significant in the sense that, code smells are not only correlated with the metrics based on their definitions but also related to other significant quality metrics such as *DIT*, *IFANIN*, *LCOM*, etc. Therefore, developers should focus on both types of metrics while improving system quality. Also, different software metrics have different impact levels on code smells. On the contrary, *comments* have a negative impact on most of the code smells, although many think *comment* itself as a smell.

on the smells such as *Anti Singleton*, *Blob*, *Complex Class*, *Large Class*, *Long Method*, and *Long Parameter List*, since these have high relationship with the *CBO* metric. In other words, if a developer wants to optimize a metric, say *CBO*, the developer should focus on the *Long Method* smell though by definition the metric is not related to the smell. Similarly, if s/he refactors a code smell, s/he will be aware of which metrics are getting optimized.

5.2 | Impactful Set of Code Smells [RQ2]

The aim of the research question (RQ2) is to identify the overall impact of each code smell based on the correlation scores with the software metrics and categorize these smells into three levels of impact—(i) *High*, (ii) *Moderate*, and (iii) *Low*. The RQ2 is different from RQ1 in the sense that, the goal of RQ2 is to measure and quantify the overall impact of the 13 code smells based on the correlation after combining the 25 software metrics, whereas RQ1 is based on direct correlation between each smell and each metric individually.

5.2.1 | Data Analysis Process for RQ2

In this study, we have measured the impact of a code smell using the coefficient of determination (r^2), which is simply the square of the correlation, r [50] to incorporate both the negative and positive correlation. Firstly, to measure the impact of a code smell, we have calculated the square of each correlation score between the code smell and each of the 25 software metrics. Then we have calculated the average r^2 score for the code smell using the same Equation (1), where in this case r is the correlation score between the smell and each software metric, and N is the total number of software metrics ($N = 25$). This average r^2 is considered as 'impact score' or simply 'impact' of the code smell in this study. In this way, the impact scores for all the 13 code smells have been measured. We have further categorized the code smells into three levels of impact—*High*, *Moderate* and *Low*. Similar to the Sub-section 5.1.2, the categories are based on the statistical quartiles [50]: quartile 1 (Q1), quartile 2 (Q2), and quartile 3 (Q3), where impact is *High* if $r^2 \geq Q3$; *Low* if $r^2 \leq Q1$; and *Moderate* if $Q3 > r^2 > Q1$. We have combined Q2 and Q3 [53] due to the relatively low number of scores in these quartiles and the minimal difference between the scores in these ranges.

5.2.2 | Result of RQ2

1. Impactful Code Smells Identification

The result based on the impact score of code smells is shown in Table 9. According to the table, firstly, it is found that five code smells are high impactful, and those are *Long Method*, *Anti Singleton*, *Complex Class*, *Large Class*, and *Long Parameter List*. Secondly, four smells have a moderate impact, and finally, the remaining four smells have a low impact. Moreover, the results provide a prioritized list of code smells based on the impact score in descending order as shown in Table 9.

Overall, the results support the findings of the previous study by Palomba et al. [2] to some extent, where they showed that the size and complexity-related code smells, such as *Long Method*, *Complex Class*, *Large Class*, etc., have a higher impact based on external software quality aspects such as fault- and change-proneness. However, there exist differences between our and their findings. For example, *Long Parameter List* has a high impact in our study, whereas they have found it as limited or low impactful. This could be because more parameters mean more code in method, resulting in creating complexity in that method. In addition, *Speculative Generality*, *Spaghetti Code* and *Refused Parent Bequest* have moderate impact in our study and high impact in their study.

The difference in impact scores between the adjacent levels seems very small. To determine whether these differences are statistically significant, we have conducted hypothesis testing similar to the Sub-section 5.1.2. Here are the hypotheses for our tests:

- *Null hypothesis (H_0): There is no significant difference in impact scores between adjacent impact levels (e.g., High vs. Moderate, Moderate vs. Low).*
- *Alternative hypothesis (H_1): There is a difference in impact scores between adjacent impact levels (e.g., High vs. Moderate, Moderate vs. Low).*

TABLE 9 | Impact of code smell based on software metrics.

Code smell	Impact score	Impact level
Long method	0.86	High
Anti singleton	0.85	
Complex class	0.85	
Large class	0.83	
Long parameter list	0.83	
Refused parent bequest	0.82	Moderate
Spaghetti code	0.82	
Speculative generality	0.80	
Blob	0.79	
Class data should be private	0.77	Low
Many field attributes but not complex	0.76	
Base class should be abstract	0.59	
Lazy class	0.26	

Results of the hypothesis testing using the one-tailed T-test are shown below:

- *High vs. Moderate Impact Levels:* p-value = 0.004 (which is less than the significance level of 0.1)
- *Moderate vs. Low Impact Levels:* p-value = 0.06 (which is less than the significance level of 0.1)

In both cases, the p-values are below the 0.1 threshold, where we consider a 10% level of significance. Therefore, we reject the null hypothesis (H_0), indicating that the differences in impact scores between the adjacent levels are statistically significant. These findings indicate that the impact scores at different levels are distinct and meaningful, providing a valid basis for categorizing the code smells into *High*, *Moderate*, and *Low* impact levels. Therefore, this analysis confirms that the impact scores serve as a discriminating factor, effectively distinguishing between the different levels of impact.

2. Code Smell Impact Based on Expert Opinion and Comparison of It With Metric Based Findings

Furthermore, we have validated our findings of the impact of code smells (Table 9) based on four experienced developers (we call them experts) from four different software companies, who have seven-plus years of experiences. We have chosen them because (i) they are highly experienced in Java software development and refactoring of code smells—41, 11, 8, and 7 plus years of experiences, respectively; (ii) they have been involved in the development of many different software systems—100, 15, 11, and 10 plus software systems. We have asked them to provide impact levels (High, Moderate and Low) on the 13 code smells based on their experiences on maintenance and/or development activities such as—solving bugs, changing codes or features, adding new features to existing software, comprehending programs, etc. The results of the impact level of code smells based on their opinion are shown in Table 10. The last column of the table is from Table 9 to visualize comparative views among the findings. The smells that match the category with the metric-based findings are labeled in bold font in the table.

According to Table 10, firstly, it is observed that out of the 5 high-impactful code smells found by our study (see Table 9), 3, 4, 3, and 2 code smells have matched with the experts, respectively. Secondly, at most 2 moderate code smells have matched with the experts, whereas these 1 or 2 matched smell types are different for each expert. Thirdly and finally, 2, 2, 3, and 2 smells have matched with the expert's opinion, respectively for the 4 low-impactful smells of the study.

From the above analysis, we have concluded the followings:

1. Our correlation-based smell impact has mostly matched with the experts for the high- and low-impactful smells.
2. It is difficult to predict moderate impactful code smells for the developers.
3. It varies a lot among the developers about the impact of code smells such as *AS*, *LPL*, *SC*, etc., though there exist many similarities for the high and low impactful smells.

TABLE 10 | Impact of code smell based on expert opinion and software metrics.

	Expert-1	Expert-2	Expert-3	Expert-4	Software metric based
High	Blob, CDSBP, CC , LC , LM	Blob, CC , LC , LM , LPL , SC	Blob, LC , CC , LPL , SC	AS, CDSBP, CC , LM	LM, AS, CC, LC, LPL
Moderate	AS, LPL, LzC, SC	CDSBP, LzC, SG	AS, CDSBP, LM, RPB	BCSBA, Blob , RPB	RPB, SC, SG, Blob
Low	BCSBA , MFABNC, RPB, SG	AS, BCSBA , MFABNC , RPB	BCSBA , LzC, MFABNC , SG	LC, LPL LzC , MFABNC , SC, SG	CDSBP, MFABNC, BCSBA, LzC

Note: Bold and plain font in code smell names indicates which has matched with our impact analysis from software metrics and which has not matched respectively.

Therefore, they should be careful about the smells, not only based on their intuition but also based on the practical impact of the smells into the systems.

However, there have been some exceptions found in our study for *Blob* and *Class Data Should Be Private* smells. Therefore, we have further investigated these exceptions. For example, the impact of *Blob* smell has not matched with any experts except for the last expert. The experts have labeled it as *High* impactful code smell whereas our finding is *Moderate*. According to the experts, it is difficult to maintain the code because of the smell's complexity, and hence it is a high-impactful smell. On the other hand, our study can not label it as *High* because of their lower frequency of occurrences in the systems. This is because developers probably avoid or refactor this smell regularly due to its high impact on maintenance. For *Class Data Should Be Private*, our study has labeled it as *Low* impact, where experts have labeled it as *Moderate* or *High*. According to the experts, this is the same as *Blob* smell that makes it difficult changing code in the maintenance phase, and also it involves with coupling. Similarly, our study can not label as the experts because of their lower frequency of occurrences in the systems.

3. Implication of the Result

Our categorization of code smells by their impact provides valuable insights for both experienced and novice developers, encouraging them to reassess the significance of different smells and prioritize their efforts to enhance software quality. This categorization supports a shift from an intuition-based to an objective-based refactoring approach during maintenance activities, leading to significant improvements in overall system quality.

By understanding which code smells have the greatest impact, developers can make more informed decisions about where to focus their refactoring efforts. This ensures that the most harmful issues are addressed first, resulting in a more efficient and effective maintenance process. Additionally, our approach helps developers identify specific metrics that are improved by addressing certain code smells, further guiding their refactoring strategies.

Overall, our work empowers developers to systematically and objectively enhance software quality, moving beyond ad-hoc fixes to a more strategic and impactful approach to code maintenance.

Summary for RQ2. Mostly size-related code smells (e.g., *Long Method*, *Large Class*) are highly impactful based on the correlation between the smells and software metrics. Structure or inheritance-related smells (e.g., *Refused Parent Bequest*, *Spaghetti Code*) have a moderate impact. Other smells have a low impact on software quality. Moreover, it varies from developer to developer about the impact level of code smells. Therefore, our general findings will help them to think more deeply about their impact on software systems.

Moreover, it helps researchers to develop automated refactoring tools based on the smell impact.

5.3 | Impact Versus Frequency of Code Smells [RQ3]

Frequencies of various code smells are not uniform for a system [4], and hence all smells do not have the same importance. Although some code smells are highly impactful, their frequencies of occurrences in the systems might not be very high. The low frequency of a smell indicates that the number of the smell's occurrences is very few. Moreover, some smells might have higher frequencies, but their impact might be very low. Therefore, these types of smells might not be very important for developers to focus on in the maintenance phase. On the other hand, highly impactful code smells having higher frequencies might be the targeted smells that should be given higher priority in the maintenance. So, only impact analysis might not be enough for developers while performing prioritization and refactoring of the smells. These cases lead us to analyze the relationship between the frequency and impact of each code smell.

5.3.1 | Data Analysis Process for RQ3

We have conducted a comprehensive analysis to measure the average smell frequency by examining the frequency of occurrences of a particular code smell across various software systems. The term 'average smell frequency' refers to the average number of times a particular code smell is found in each software system. To achieve this, first, we have identified the number of occurrences of the code smell in each of the 35 different software systems. After gathering this data, we have proceeded to calculate the average smell frequency for each code smell using the

following formula (Equation 2):

$$\text{Average Smell Frequency, ASF} = \frac{\sum_{i=1}^N F_i}{N} \quad (2)$$

where F_i represents the frequency of the code smell in each individual system, and N represents the total number of systems, which in our study is 35.

By applying this formula, we were able to calculate the average smell frequency for each of the 13 different code smells under consideration. Furthermore, similar to the methodology described in Sub-section 5.2.1, we have categorized the code smells into the three levels of frequency—*High*, *Moderate*, and *Low*. This categorization is based on statistical quartiles, as described by Gupta and Kapoor [50]. We have used the first quartile (Q1), the second quartile (Q2), and the third quartile (Q3) to determine these levels of frequency. Specifically, we defined the frequency levels as follows:

TABLE 11 | Average frequency level of code smell.

Code smell	Average smell frequency (Per project)	Frequency level
Long method	733.06	High
Complex class	565.06	
Long parameter list	245.77	
Large class	231.60	
Class data should be private	110.60	Moderate
Anti singleton	86.34	
Lazy class	79.03	
Blob	20.91	
Base class should be abstract	8.74	Low
Refused parent bequest	7.91	
Spaghetti code	7.17	
Many field attributes but not complex	4.57	
Speculative generality	3.14	

- *High Frequency*: Metrics with *ASF* values greater than or equal to Q3, representing the top 25% of the *ASF* scores.
- *Low Frequency*: Metrics with *ASF* values less than or equal to Q1, representing the bottom 25% of the *ASF* scores.
- *Moderate Frequency*: Metrics with *ASF* values between Q1 and Q3, representing the middle 50% of the *ASF* scores.

This detailed approach ensures a precise and meaningful analysis of code smell frequencies across the software systems, allowing us to better understand the prevalence and distribution of the code smells.

5.3.2 | Result of RQ3

1. Frequency Level of Code Smells

The result based on the average frequency of code smells is shown in Table 11. According to the table, firstly, it is found that four code smells have high frequency: *Long Method*, *Complex Class*, *Long Parameter List*, and *Large Class*. Secondly, the five moderately frequent smells are *Class Data Should Be Private*, *Anti Singleton*, *Lazy Class*, *Blob*, and *Base Class Should Be Abstract*. Finally, the four low frequent smells are – *Refused Parent Bequest*, *Spaghetti Code*, *Many Field Attributes But Not Complex* and *Speculative Generality*. The frequency category will help developers in understanding about which smells occur more or less.

2. Relationship Between Frequency and Impact of Code Smells

After identifying the frequency levels of the code smells, it is necessary to analyze whether frequency differs from impact or not. Therefore, we have compared our analysis between the smell frequency and impact for each smell. The comparative result of the analysis is summarized in the following Table 12. According to the table, firstly, it is observed that four high-frequent smells, namely *Long Method*, *Complex Class*, *Large class*, and *Long Parameter List*, have a high impact on software quality. So these smells should be carefully monitored while developing and maintaining a system. Only *Blob* and *Many Field Attributes But Not Complex* smells have both moderate and low impact and

TABLE 12 | Impact versus frequency of code smell.

		Frequency level		
		High	Moderate	Low
Impact	High	Long method, Complex class, Large class, Long parameter list	Anti singleton	
	Moderate		Blob	Refused parent bequest, Spaghetti code, Speculative generality
	Low		Class data should be private, Base class should be abstract, LazyClass	Many field attributes but not complex

Summary for RQ3. One can view that high, moderate, and low frequency of code smells mean high, moderate, and low impact on software metrics, respectively. These cases do not always hold for all the smell types. For example, *RPB*, *SC*, and *SG* have a moderate impact, though their frequency level is low, whereas *AS* has a high impact with a moderate frequency level.

frequency level respectively. Secondly, we have observed several exceptions in the relationships. For instance, it has been seen that *Anti Singleton* smell has a moderate frequency level, though its impact level is high. In addition, three smells – *RPB*, *SC* and *SG* – have low frequency with moderate impact, whereas three other smells *CDSBP*, *BCSBA* and *LzC* – have moderate frequency with low impact. These exceptions have concluded that it is not always true that high, moderate, and low frequency of code smells indicate high, moderate and low impact, respectively. That is, not all of the smell frequency levels have the same kind of effect on the correlation-based impact. To make it clear, the correlations show how the software metrics of a smell type will increase (or decrease) when the smell frequency increases (or decrease). On the other hand, frequency just tells their frequency of occurrences in the systems.

3. Implication of the Result

This comparative analysis between the impact and frequency of code smells adds new contribution to the research field by providing meaningful insights to the developers and researchers. The result suggests they focus on the high-impactful smells having a higher frequency level while developing software and innovating refactoring tools. On the other hand, they can focus less on the lower-impactful smells having any frequency levels as their impact can be negligible. Moreover, they can avoid low-impactful smells with low frequency levels to save development and maintenance time.

6 | Threats to Validity

The construct, external, internal, and reliability threats to validity [54] of this empirical study are discussed in this section.

1. Threats to Construct Validity

The threats concern the relationship between theory and observation due to measurement errors in our study. The potential threats are related to the accuracy of the tools used in this study to detect code smells from software systems. The results of the tool might lead to a different phenomenon for different tools than our investigation. However, we have used the tool DÉCOR, which has been evaluated in previous studies as having good accuracy, and hence it might mitigate the threats to some extent.

2. Threats to External Validity

Regarding the generalization of our findings, we limited our study based on only Java systems, due to limitations of the infrastructure we used (e.g., for a large number of code smells, detection tools only work on Java systems). Moreover, the choice of

projects from Apache and Eclipse codebases is also a threat to validity. However, these projects are well-known in this research domain, and one can easily use them for replication and comparison purposes. In addition, the impact of code smells is based on the 25 internal software metrics. However, this is the largest study in terms of the number of software metrics (25) and considered code smell types (13) on the impact analysis of code smells on software metrics. Moreover, we focused on open-source systems only, and we cannot speculate about how the results would be different for industrial systems. Since the frequency of code smells between open-source and industrial systems was observed by Rahman et al. [4], these results can be generalized to some extent.

3. Threats to Internal Validity

The threats concern factors that could influence our observations. We are aware that we cannot claim a direct cause-effect relationship between the occurrence of code smells and software metrics. In particular, our observations may be influenced by the different factors related to development phases (e.g., experience of developers, workload, etc.). We focused on the fact that some smells are highly associated with some of the metrics, while some are not.

4. Threats to Reliability Validity

The potential threats concern the possibility of replicating the study. However, we have provided all the necessary details, including the synthesized dataset, to replicate our findings (see Section 4).

7 | Discussion and Conclusion

We conducted a large study on 35 Java open-source systems, with the purpose of understanding the impact of code smells on the systems and their relation with internal software metrics. The results highlighted the following findings:

- **Relationship between code smell and software metric:** Most of the code smells have high correlation with size-related metrics such as the number of files, classes, methods, statements, etc. This means that when the occurrence of the smells increases, these metric scores also increase, and hence it decreases maintainability. Moreover, a code smell (e.g., *Long Method*) is not only associated with the metrics (e.g., *size*) by which it is defined, but also other metrics (such as *DIT*, *CBO*, *CC*, etc.) that are significant. On the other hand, same metric can be correlated with multiple smells, and hence developers should focus on these smells while optimizing the particular metric. For instance, if the goal of a system is to reduce coupling, the developers should focus on the code smells related to the *CBO* metric, such as *Anti Singleton*, *Blob*, *Complex Class*, *Large Class*, *Long Method*, and *Long Parameter List*. Therefore, the relevant and impactful metrics might be useful for developers to optimize the metrics through smell refactoring according to the system's need to improve the maintainability. At the same time, the developers can recognize which metrics are going to be optimized while refactoring a particular smell. These findings help them in objective-based

smell-refactoring activities, as in objective-based maintenance activities [3].

- **Impact of code smell:** The three categories of impact level – *High*, *Moderate*, and *Low* – for the code smells provide insights to developers about which code smells should be focused on while writing code and developing a system. For example, the developers should be careful about the highly impactful smells, such as *Long Method*, *Anti Singleton*, *Complex Class*, *Large Class*, and *Long Parameter List*. Therefore, the quality of the system will be improved from the beginning of the system's life cycle, which results in increasing maintainability in the long run. In addition, it is difficult for the developers to work with a large number of smells. So, the short list of impactful smells helps them to prioritize refactoring tasks for the smells based on their impact and the system's need.
- **Relation between frequency and impact of code smell:** Since all code smells do not occur with similar frequencies, developers might not need to focus on all of these. For instance, smells with both low frequency and low impact, such as *Many Field Attributes But Not Complex*, can be avoided, as low frequency means that the overall impact of the smell on the system is not very significant. On the contrary, smells having both high frequency and high impact, such as *Long Method*, *Complex Class*, *Large Class*, and *Long Parameter List*, should be given higher priority due to their overall high impact on the system. Moreover, smells with moderate frequency but high impact, like *Anti Singleton*, should also be given high priority while maintaining the system.

Our results clearly show that code smells should be carefully monitored by developers, since these smells are related to a lot of software quality metrics such as size, coupling, complexity, etc. It is significant in a sense that code smells are correlated with both smell definition and non-definition-based metrics. This, in fact, helps developers to prioritize refactoring activities of the smells while optimizing a set of metrics of a system. Our results also conclude that all the smells do not have the same impact on software systems, and some smells are highly impactful, having a higher frequency. The shortlist of the most impactful smells having higher frequency can help to write or reuse source code carefully without inducing these smells from the beginning of software development. Moreover, our findings not only help practitioners but also researchers to develop automated refactoring tools based on the impact of code smells and prioritize the smells.

As for future research, we will focus on external software metrics such as maintainability, understandability, etc., to measure the impact of code smells and prioritize these. Moreover, we plan to further analyze other factors such as developers' experience, workload, etc., which influence the software quality metrics.

Author Contributions

The author takes full responsibility for this article.

Acknowledgments

We gratefully acknowledge the funding support of the Information and Communication Technology (ICT) Division, Ministry of Posts, Telecommunications and Information Technology, Dhaka, Bangladesh, through Grant/Fellowship Number: 56.00.0000.052.33.005.21-2, 18-01-2022.

Conflicts of Interest

The authors declare no conflicts of interest.

Data Availability Statement

The datasets generated during and/or analyzed during the current study are available in the repository: <https://bitbucket.org/masudshrabon/impactstudy/>.

Endnotes

¹ <https://www.apache.org/>.

² <https://www.eclipse.org/org/>.

³ <https://www.scitools.com/>.

⁴ <https://bitbucket.org/masudshrabon/impactstudy/>.

⁵ <https://bitbucket.org/masudshrabon/impactstudy/>.

⁶ Link of rule cards: <https://github.com/ptidejteam/ptidej-Ptidej/tree/master/SAD%20Rules%20Creator/src/main/resources>, Last Accessed on 14 June, 2024.

References

1. M. Fowler, K. Beck, J. Brant, W. Opdyke, and D. Roberts, *Refactoring: Improving the Design of Existing Code* (Reading, MA: Addison-Wesley, 1999).
2. F. Palomba, G. Bavota, M. D. Penta, F. Fasano, R. Oliveto, and A. D. Lucia, "On the Diffuseness and the Impact on Maintainability of Code Smells: A Large Scale Empirical Investigation," *Empirical Software Engineering* 23, no. 3 (2018): 1188–1221.
3. P. Tripathy and K. Naik, *Software Evolution and Maintenance: A practitioner's Approach* (Hoboken, NJ: John Wiley & Sons, 2014).
4. M. M. Rahman, A. Satter, M. M. A. Joarder, and K. Sakib, "An Empirical Study on the Occurrences of Code Smells in Open Source and Industrial Projects," in *Proceedings of the 16th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement*, (2022); 289–294.
5. G. Bavota, A. De Lucia, M. Di Penta, R. Oliveto, and F. Palomba, "An Experimental Investigation on the Innate Relationship Between Quality and Refactoring," *Journal of Systems and Software* 107 (2015): 1–14.
6. A. Kaur, "A Systematic Literature Review on Empirical Analysis of the Relationship Between Code Smells and Software Quality Attributes," *Archives of Computational Methods in Engineering* 27, no. 4 (2020): 1267–1296.
7. R. Baggen, J. P. Correia, K. Schill, and J. Visser, "Standardized Code Quality Benchmarking for Improving Software Maintainability," *Software Quality Journal* 20, no. 2 (2012): 287–307.
8. F. Zhang, A. Mockus, Y. Zou, F. Khomh, and A. E. Hassan, "How Does Context Affect the Distribution of Software Maintainability Metrics?," in *International Conference on Software Maintenance (ICSM)* (New York, NY: IEEE, 2013), 350–359.
9. M. Alshayeb, "Empirical Investigation of Refactoring Effect on Software Quality," *Information and Software Technology* 51, no. 9 (2009): 1319–1326.
10. F. Khomh, M. D. Penta, Y. G. Guéhéneuc, and G. Antoniol, "An Exploratory Study of the Impact of Antipatterns on Class Change-and Fault-Proneness," *Empirical Software Engineering* 17, no. 3 (2012): 243–275.

11. M. Abbes, F. Khomh, Y. G. Gueheneuc, and G. Antoniol, "An Empirical Study of the Impact of Two Antipatterns, Blob and Spaghetti Code, on Program Comprehension," in *Proceedings of the 15th European Conference on Software Maintenance and Reengineering, IEEE*, (2011); 181–190.
12. N. Sae-Lim, S. Hayashi, and M. Saeki, "How Do Developers Select and Prioritize Code Smells? A Preliminary Study," in *International Conference on Software Maintenance and Evolution (ICSME), IEEE*, (2017); 484–488.
13. F. Pecorelli, F. Palomba, F. Khomh, and A. De Lucia, "Developer-Driven Code Smell Prioritization," in *Proceedings of the 17th International Conference on Mining Software Repositories (MSR)*, 2020; 220–231.
14. S. A. Vidal, C. Marcos, and J. A. Díaz-Pace, "An Approach to Prioritize Code Smells for Refactoring," *Automated Software Engineering* 23, no. 3 (2016): 501–532.
15. N. Sae-Lim, S. Hayashi, and M. Saeki, "Context-Based Approach to Prioritize Code Smells for Prefactoring," *Journal of Software: Evolution and Process* 30, no. 6 (2018): e1886.
16. F. Sabir, F. Palma, G. Rasool, Y. G. Guéhéneuc, and N. Moha, "A Systematic Literature Review on the Detection of Smells and Their Evolution in Object-Oriented and Service-Oriented Systems," *Software: Practice and Experience* 49, no. 1 (2019): 3–39.
17. N. Tsantalis, T. Chaikalas, and A. Chatzigeorgiou, "Ten Years of Jdeodorant: Lessons Learned From the Hunt for Smells," in *Proceedings of the 25th International Conference on Software Analysis, Evolution and Reengineering (SANER), IEEE*, (2018); 4–14.
18. N. Moha, Y. G. Guéhéneuc, L. Duchien, and A. F. Le Meur, "Decor: A Method for the Specification and Detection of Code and Design Smells," *IEEE Transactions on Software Engineering* 36, no. 1 (2009): 20–36.
19. F. Palomba, G. Bavota, M. Di Penta, R. Oliveto, D. Poshyvanyk, and A. De Lucia, "Mining Version Histories for Detecting Code Smells," *IEEE Transactions on Software Engineering* 41, no. 5 (2014): 462–489.
20. I. Iso, *Iec 9126-1: Software Engineering-Product Quality-Part 1: Quality Model* (Geneva, Switzerland: International Organization for Standardization, 2001), 21.
21. S. Kaur and R. Maini, "Analysis of Various Software Metrics Used to Detect Bad Smells," *International Journal of Engineering & Science (IJES)* 5, no. 6 (2016): 14–20.
22. B. L. Sousa, M. A. Bigonha, and K. A. Ferreira, "An Exploratory Study on Cooccurrence of Design Patterns and Bad Smells Using Software Metrics," *Software: Practice and Experience* 49, no. 7 (2019): 1079–1113.
23. F. Khomh, M. Di Penta, and Y. G. Gueheneuc, "An Exploratory Study of the Impact of Code Smells on Software Change-Proneess," in *Proceedings of the 16th Working Conference on Reverse Engineering, IEEE*, (2009); 75–84.
24. W. Li and R. Shatnawi, "An Empirical Study of the Bad Smells and Class Error Probability in the Post-Release Object-Oriented System Evolution," *Journal of Systems and Software* 80, no. 7 (2007): 1120–1128.
25. S. M. Olbrich, D. S. Cruzes, and D. I. Sjøberg, "Are all Code Smells Harmful? A Study of God Classes and Brain Classes in the Evolution of Three Open Source Systems," in *International Conference on Software Maintenance (ICSME), IEEE*, 2010; 1–10.
26. A. Saboury, P. Musavi, F. Khomh, and G. Antoniol, "An Empirical Study of Code Smells in Javascript Projects," in *Proceedings of the 24th International Conference on Software Analysis, Evolution and Reengineering (SANER), IEEE*, (2017); 294–305.
27. Y. Zhong, M. Shi, J. He, C. Fang, and Z. Chen, "Security-Based Code Smell Definition, Detection, and Impact Quantification in Android," *Software: Practice and Experience* 53, no. 11 (2023): 2296–2321.
28. M. M. Rahman, T. Ahammed, M. M. A. Joarder, and K. Sakib, "Does Code Smell Frequency Have a Relationship With Fault-Proneess?," in *Proceedings of the 27th International Conference on Evaluation and Assessment in Software Engineering*, (2023); 261–262.
29. M. M. Rahman, R. R. Riyadh, S. M. Khaled, A. Satter, and M. R. Rahman, "Mmruc3: A Recommendation Approach of Move Method Refactoring Using Coupling, Cohesion, and Contextual Similarity to Enhance Software Design," *Software: Practice and Experience* 48, no. 9 (2018): 1560–1587.
30. A. Yamashita and L. Moonen, "Exploring the Impact of Inter-Smell Relations on Software Maintainability: An Empirical Study," in *Proceedings of the 35th International Conference on Software Engineering (ICSE), IEEE*, 2013; 682–691.
31. A. Yamashita, M. Zanoni, F. A. Fontana, and B. Walter, "Inter-Smell Relations in Industrial and Open Source Systems: A Replication and Comparative Analysis," in *International Conference on Software Maintenance and Evolution (ICSME), IEEE*, 2015; 121–130.
32. F. Palomba, G. Bavota, M. Di Penta, F. Fasano, R. Oliveto, and A. De Lucia, "A Large-Scale Empirical Study on the Lifecycle of Code Smell Co-Occurrences," *Information and Software Technology* 99 (2018): 1–10.
33. J. Martins, C. Bezerra, A. Uchôa, and A. Garcia, "Are Code Smell Co-Occurrences Harmful to Internal Quality Attributes? A Mixed-Method Study," in *Proceedings of the 34th Brazilian Symposium on Software Engineering*, (2020); 52–61.
34. J. Martins, C. Bezerra, A. Uchôa, and A. Garcia, "How Do Code Smell Co-Occurrences Removal Impact Internal Quality Attributes? A Developers' Perspective," *Brazilian Symposium on Software Engineering* (2021): 54–63.
35. D. I. Sjøberg, A. Yamashita, B. C. Anda, A. Mockus, and T. Dybå, "Quantifying the Effect of Code Smells on Maintenance Effort," *IEEE Transactions on Software Engineering* 39, no. 8 (2012): 1144–1156.
36. F. Palomba, G. Bavota, M. Di Penta, R. Oliveto, and A. De Lucia, "Do They Really Smell Bad? A Study on Developers' Perception of Bad Code Smells," in *International Conference on Software Maintenance and Evolution (ICSME), IEEE*, (2014); 101–110.
37. D. Taibi, A. Janes, and V. Lenarduzzi, "How Developers Perceive Smells in Source Code: A Replicated Study," *Information and Software Technology* 92 (2017): 223–235.
38. R. Verma, K. Kumar, and H. K. Verma, "Code Smell Prioritization in Object-Oriented Software Systems: A Systematic Literature Review," *Journal of Software: Evolution and Process* 35, no. 12 (2023): e2536.
39. S. Vidal, H. Vazquez, J. A. Diaz-Pace, C. Marcos, A. Garcia, and W. Oizumi, "Jspirit: A Flexible Tool for the Analysis of Code Smells," in *Proceedings of the 34th International Conference of the Chilean Computer Science Society (SCCC), IEEE*, 2015; 1–6.
40. A. Ouni, M. Kessentini, S. Bechikh, and H. Sahraoui, "Prioritizing Code-Smells Correction Tasks Using Chemical Reaction Optimization," *Software Quality Journal* 23, no. 2 (2015): 323–361.
41. M. R. Islam, A. Al Maruf, and T. Cerny, "Code Smell Prioritization With Business Process Mining and Static Code Analysis: A Case Study," *Electronics* 11, no. 12 (2022): 1880.
42. T. Guggulothu and S. A. Moiz, "An Approach to Suggest Code Smell Order for Refactoring," in *International Conference on Emerging Technologies in Computer Engineering* (Berlin/Heidelberg, Germany: Springer, 2019), 250–260.
43. A. Gupta and N. K. Chauhan, "A Severity-Based Classification Assessment of Code Smells in Kotlin and Java Application," *Arabian Journal for Science and Engineering* 47, no. 2 (2022): 1831–1848.
44. F. A. Fontana, V. Ferme, M. Zanoni, and R. Roveda, "Towards a Prioritization of Code Debt: A Code Smell Intensity Index," in *Proceedings of the 7th International Workshop on Managing Technical Debt (MTD), IEEE*, (2015); 16–24.

45. A. Oliveira, L. Sousa, W. Oizumi, and A. Garcia, "On the Prioritization of Design-Relevant Smelly Elements: A Mixed-Method, Multi-Project Study," in *Proceedings of the 13th Brazilian Symposium on Software Components, Architectures, and Reuse*, (2019); 83–92.
46. V. Lenarduzzi, T. Besker, D. Taibi, A. Martini, and F. A. Fontana, "A Systematic Literature Review on Technical Debt Prioritization: Strategies, Processes, Factors, and Tools," *Journal of Systems and Software* 171, no. 110 (2021): 827.
47. F. Pecorelli, F. Palomba, D. Di Nucci, and A. De Lucia, "Comparing Heuristic and Machine Learning Approaches for Metric-Based Code Smell Detection," in *Proceedings of the 27th International Conference on Program Comprehension (ICPC)*, *IEEE*, (2019); 93–104.
48. M. Tufano, F. Palomba, G. Bavota, et al., "When and Why Your Code Starts to Smell Bad," in *ACM 37th IEEE International Conference on Software Engineering*, vol. 1, (2015); 403–414.
49. C. Zhu, X. Zhang, Y. Feng, and L. Chen, "An Empirical Study of the Impact of Code Smell on File Changes," in *International Conference on Software Quality, Reliability and Security (QRS)*, *IEEE*, 2018; 238–248.
50. S. Gupta and V. Kapoor, *Fundamentals of Mathematical Statistics* (New Delhi, India: Sultan Chand & Sons, 2020).
51. M. J. Crawley, *The R Book* (Hoboken, NJ: John Wiley & Sons, 2012).
52. P. Schober, C. Boer, and L. A. Schwarte, "Correlation Coefficients: Appropriate Use and Interpretation," *Anesthesia & Analgesia* 126, no. 5 (2018): 1763–1768.
53. K. K. Arkema, G. Guannel, G. Verutes, et al., "Coastal Habitats Shield People and Property From Sea-Level Rise and Storms," *Nature Climate Change* 3, no. 10 (2013): 913–918.
54. P. Runeson, M. Host, A. Rainer, and B. Regnell, *Case Study Research in Software Engineering: Guidelines and Examples* (Hoboken, NJ: John Wiley & Sons, 2012).

Appendix A

TABLE A1 | Correlation between code smell and software metric.

Smell vs. Metric	AS	BCSBA	Blob	CDSBP	CC	LC	LzC	LM	LPL	MFABNC	RPB	SC	SG
NOM	0.95993	0.788256	0.937179	0.948287	0.951327	0.962673	0.483065	0.958263	0.933613	0.900372	0.955154	0.960633	0.913428
NDM	0.765655	0.56078	0.668236	0.747574	0.823843	0.799832	0.435835	0.797871	0.785629	0.326087	0.42563	0.731622	0.653798
NPriM	0.919742	0.744476	0.882708	0.840771	0.945931	0.952028	0.40654	0.912605	0.916247	0.700166	0.856722	0.852808	0.734058
NProM	0.910913	0.678148	0.876847	0.916398	0.939141	0.941242	0.409185	0.905246	0.929337	0.245335	0.800463	0.746629	0.770659
NPM	0.956974	0.768118	0.921834	0.911514	0.946285	0.952448	0.485103	0.953782	0.92465	0.890899	0.951375	0.954427	0.907678
NOS	0.951664	0.787198	0.91416	0.920269	0.942783	0.950067	0.528148	0.953782	0.919048	0.975648	0.956743	0.976299	0.936965
NDS	0.961572	0.800902	0.939539	0.955073	0.962673	0.962673	0.558332	0.972269	0.945378	0.975335	0.960973	0.98292	0.941023
NExS	0.936501	0.770957	0.895742	0.895425	0.926395	0.939141	0.452463	0.934454	0.89972	0.858979	0.953873	0.966899	0.890998
CC	0.931899	0.773449	0.91109	0.830962	0.9362	0.947826	0.463697	0.936415	0.90112	0.889365	0.952967	0.937418	0.87261
LCOM	0.987519	0.84827	0.96618	0.881416	0.960992	0.939422	0.356181	0.969188	0.968347	0.899632	0.9027	0.974163	0.916218
DIT	0.979196	0.829827	0.946988	0.975312	0.953985	0.890819	0.618322	0.964986	0.955182	0.99984	0.955363	0.966313	0.972103
IFANIN	0.969664	0.855465	0.94719	0.919987	0.939702	0.891379	0.597388	0.970588	0.968695	1	0.943133	0.960536	0.970592
CBO	0.924897	0.749326	0.913185	0.898012	0.940262	0.908747	0.418453	0.956229	0.940542	0.452768	0.727641	0.859731	0.803694
NOCh	0.86587	0.701314	0.654922	0.886507	0.941943	0.85269	0.538111	0.904799	0.905983	NA	0.959653	0.470353	0.798877
RFC	0.942085	0.75103	0.93477	0.844744	0.944184	0.938161	0.497541	0.938375	0.917367	0.904775	0.950547	0.937057	0.917815
NIM	0.952814	0.771167	0.936196	0.882401	0.948526	0.940822	0.509207	0.954902	0.929132	0.906863	0.941502	0.948242	0.921533
NIV	0.907424	0.791684	0.93181	0.897312	0.93634	0.945444	0.426782	0.938095	0.916247	0.791311	0.920292	0.892523	0.911426
WMC	0.958726	0.786196	0.937511	0.87879	0.953568	0.93676	0.501055	0.957143	0.936695	0.904775	0.955082	0.954929	0.920548
NOC	0.979853	0.850312	0.948671	0.978655	0.949366	0.95651	0.608104	0.97864	0.963233	0.99992	0.951519	0.966437	0.968896
NOF	0.999836	0.890923	0.991162	0.949048	0.99916	1	0.676597	0.99972	1	1	0.984073	0.999928	0.985118
NL	0.953088	0.768953	0.904951	0.844095	0.943483	0.943203	0.597782	0.947619	0.923529	0.982188	0.956743	0.965333	0.941628
BLOC	0.958999	0.756917	0.920624	0.941502	0.931158	0.943343	0.572091	0.947059	0.917367	0.965366	0.959462	0.961487	0.938923
LOC	0.948927	0.792108	0.904293	0.845628	0.941943	0.948106	0.544574	0.944258	0.922409	0.981551	0.959613	0.971884	0.944934
NCL	0.914714	0.759658	0.864442	0.889515	0.845437	0.841796	0.620946	0.884874	0.87535	0.97701	0.922606	0.915627	0.936669
RCTC	−0.25079	0.460177	0.176445	−0.21869	−0.37214	−0.34324	−0.22999	−0.33001	−0.26962	0.77801	0.696033	0.476276	0.723828